

TBITW ML Library

Implementation Details

dansyc11

Last Updated: May 11, 2026

1 Introduction

I built this library understand ML from first principle with just C

1.1 Core Components

- Arena allocator for fast memory management
- Matrix operations for linear algebra
- Neural network with backpropagation
- Adam optimizer for training

2 Memory Management

2.1 Arena Allocator

Pre-allocate large block, bump pointer for $O(1)$ allocation:

```
1 typedef struct {  
2     u8 *memory;  
3     usize size;  
4     usize used;  
5 } Arena;
```

Listing 1: Arena Structure

Eliminates malloc/free overhead during training.

Operation	malloc/free	Arena
Single Allocation	$O(\log n)$	$O(1)$
Batch Deallocation	$O(n)$	$O(1)$

Table 1: Allocator Performance

3 Neural Network

3.1 Architecture

```
1 typedef struct {  
2     Mat *ws;      // Weights [L]  
3     Mat *bs;      // Biases [L]  
4     Mat *as;      // Activations [L+1]  
5     u32 count;  
6 } NN;
```

Listing 2: Network Structure

3.2 Forward Pass

For layer l :

$$z^{(l)} = a^{(l-1)}W^{(l)} + b^{(l)} \quad (1)$$

$$a^{(l)} = \sigma(z^{(l)}) \quad (2)$$

Where sigmoid: $\sigma(x) = \frac{1}{1+e^{-x}}$

3.3 Cost Function

Mean squared error:

$$J = \frac{1}{N} \sum_{i=1}^N \sum_{j=1}^{n_L} (a_j^{(L)} - y_j)^2 \quad (3)$$

4 Backpropagation

Output layer gradient:

$$\delta^{(L)} = 2(a^{(L)} - y) \odot \sigma'(z^{(L)}) \quad (4)$$

Hidden layer gradient:

$$\delta^{(l)} = (\delta^{(l+1)}(W^{(l+1)})^T) \odot \sigma'(z^{(l)}) \quad (5)$$

Weight updates:

$$\frac{\partial J}{\partial W^{(l)}} = (a^{(l-1)})^T \delta^{(l)} \quad (6)$$

5 Adam Optimizer

Maintains momentum m_t and adaptive learning rate v_t .

Hyperparameters: $\beta_1 = 0.9$, $\beta_2 = 0.999$, $\epsilon = 10^{-8}$, $\alpha = 0.001$

Update rules:

$$m_t = \beta_1 m_{t-1} + (1 - \beta_1) g_t \quad (7)$$

$$v_t = \beta_2 v_{t-1} + (1 - \beta_2) g_t^2 \quad (8)$$

$$\hat{m}_t = \frac{m_t}{1 - \beta_1^t}, \quad \hat{v}_t = \frac{v_t}{1 - \beta_2^t} \quad (9)$$

$$\theta_{t+1} = \theta_t - \alpha \frac{\hat{m}_t}{\sqrt{\hat{v}_t} + \epsilon} \quad (10)$$

I implemented Adam to try fixing the XOR convergence problem. It didn't improve XOR performance significantly, but it did increase accuracy on both MNIST digits and Fashion-MNIST by 2-3% compared to vanilla SGD.

6 Training

6.1 Xavier Initialization

$$W^{(l)} \sim U \left[-\sqrt{\frac{6}{n_{l-1} + n_l}}, \sqrt{\frac{6}{n_{l-1} + n_l}} \right] \quad (11)$$

Prevents gradient vanishing/explosion.

6.2 Configuration

Parameter	Value
Learning Rate	0.001
Batch Size	100
Epochs	20
Architecture	784-128-10

7 Results

7.1 MNIST Performance

Epoch	Train Loss	Test Accuracy
1	0.245	45.50%
5	0.089	87.67%
10	0.032	90.68%
20	0.018	92.37%

Table 2: MNIST Training Progression

7.2 Fashion-MNIST

Final accuracy: 92.16% on 10 clothing categories.

8 ASCII Visualization

The prediction tools visualize 28x28 images in the terminal using ASCII characters. Each pixel value (0.0 to 1.0) is mapped to a character based on intensity:

```
1 void print_digit(Mat image) {
2     const char *chars = "░.°0@";
3     for (u32 i = 0; i < 28; ++i) {
4         for (u32 j = 0; j < 28; ++j) {
5             f32 pixel = image.data[i * 28 + j];
6             int idx = (int)(pixel * 5);
7             printf("%c", chars[idx]);
8         }
9         printf("\n");
10    }
11 }
```

Listing 3: ASCII Visualization

Darker pixels (near 0.0) map to spaces, while brighter pixels (near 1.0) map to '@' symbols. This creates a recognizable visualization of handwritten digits and clothing items directly in the terminal.

9 Model File Format

Binary format:

1. Magic: 0x54424954 ("TBIT")
2. Layer count: u32
3. For each layer: dimensions + weights + biases

File size: ~407 KB for 784-128-10 network.

10 Future Improvements

- ☐ ReLU activation to improve gradient flow
- ☐ Convolutional layers to maximize MNIST accuracy (target: 98-99%)
- ☐ Test on additional datasets beyond MNIST/Fashion-MNIST

11 Appreciation

- **Tsoding** - ML in C series on YouTube. Core implementation techniques and educational approach taken from his tutorials on building neural networks from scratch.
- **Magicalbat** - Additional neural network implementation guidance and best practices.

Implementation written from scratch with educational guidance from the above resources.